MyRedis

In-Memory Database Server

Complete Technical Deep Dive

Architecture | Implementation | Networking | Concurrency

Language: C++17 | ~1,573 Lines of Code | 27 Commands Supported

Networking: POSIX TCP Sockets | RESP Wire Protocol

Concurrency: Thread-per-Client | std::mutex Synchronization

Data Structures: unordered_map (O(1) lookup) | vector | nested maps

Persistence: Custom text-based dump format (dump.my_rdb)

Compatible with official redis-cli

3 Classes | 26 Database Methods | 38 Automated Tests

Author: Mandar

1. What Is This Project?

> *A Redis-compatible in-memory database server built entirely from scratch in C++17. It listens on a TCP port, accepts multiple client connections simultaneously, parses the Redis RESP wire protocol, stores data in memory using hash maps and vectors, supports 27 Redis commands across strings, lists, and hashes, persists data to disk periodically and on shutdown, and handles graceful shutdown via Unix signals. You can connect to it using the official redis-cli tool -- it speaks the same protocol as real Redis.*

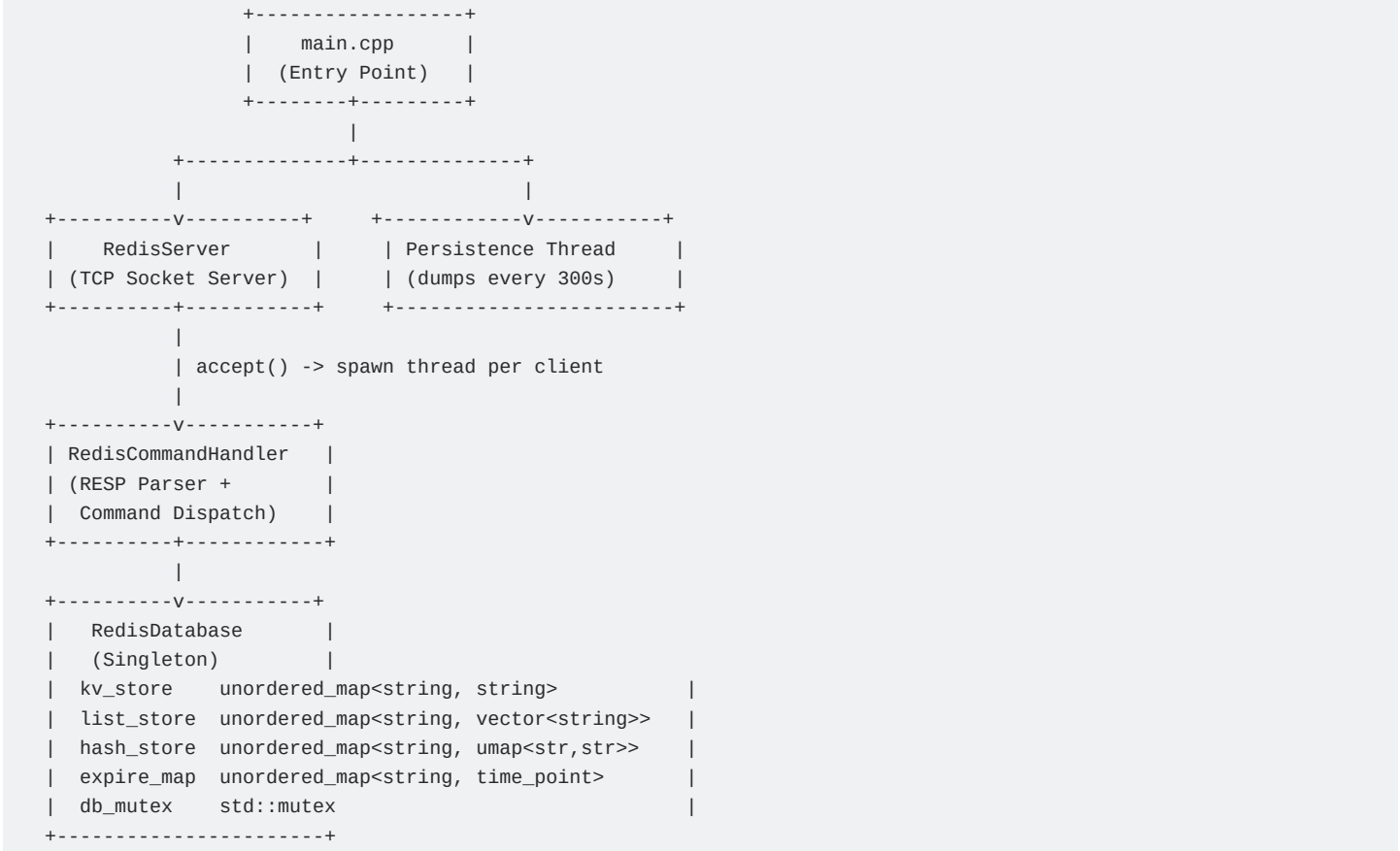
What Is Redis and Why Clone It?

Redis (Remote Dictionary Server) is the world's most popular in-memory data store, used as a cache, message broker, and session store.

Building a Redis clone teaches core systems programming skills:

- How network servers work at the lowest level (sockets, TCP)
- How to handle multiple clients simultaneously (threads, mutexes)
- How to design an efficient in-memory data store
- How protocols work (parsing binary wire formats)
- How persistence works (serializing memory to disk)

2. Architecture Overview



Three Classes, Clean Separation

Class	Responsibility
RedisServer	Owns TCP socket, accepts clients, spawns threads
RedisCommandHandler	Parses RESP protocol, dispatches to handler functions
RedisDatabase	Stores all data, thread-safe via mutex, handles persistence

Entry Point (main.cpp) - 27 Lines

```
main()
  |
  +-- Parse port from argv (default 6379)
  +-- Load database from dump.my_rdb (if exists)
  +-- Create RedisServer(port)
  +-- Spawn persistence thread (dumps every 300s, detached)
  +-- server.run()  <-- blocks here until shutdown
```

3. TCP/IP and Socket Programming

The Server Socket Lifecycle

```
1. socket()    -- Create a socket (get a phone)
2. bind()      -- Assign address + port (get a phone number)
3. listen()    -- Start accepting calls (turn the ringer on)
4. accept()    -- Wait for a client to connect (pick up the phone)
5. recv/send   -- Exchange data with the client (talk)
6. close()     -- Hang up
```

Implementation Details

```
// Create socket
server_socket = socket(AF_INET, SOCK_STREAM, 0);
// AF_INET = IPv4, SOCK_STREAM = TCP

// Allow port reuse (avoid 'Address already in use')
int opt = 1;
setsockopt(server_socket, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

// Bind to all interfaces on our port
sockaddr_in serverAddr{};
serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(port);  // host-to-network byte order
bind(server_socket, (struct sockaddr*)&serverAddr, sizeof(serverAddr));

// Start listening (backlog of 5 pending connections)
listen(server_socket, 5);

// Accept loop
while (running) {
    int client_socket = accept(server_socket, nullptr, nullptr);
    // spawn a thread to handle this client...
}
```

Key Networking Concepts

htons(): Converts port from host byte order to network byte order (big-endian). Ensures both sides agree on the byte layout.

SO_REUSEADDR: When you kill a server, the OS keeps the port in TIME_WAIT for ~60s. This option lets us rebind immediately on restart.

Backlog: Number of pending connections the OS queues before refusing new ones.

4. RESP Protocol

RESP (REdis Serialization Protocol) is how redis-cli talks to the server. It's a text-based, binary-safe protocol.

Data Types

Prefix	Type	Example	Meaning
+	Simple String	+OK\r\n	Success message
-	Error	-Error: key not found\r\n	Error message
:	Integer	:42\r\n	Numeric result
$	Bulk String	$5\r\nhello\r\n	Length-prefixed string
$-1	Null	$-1\r\n	Key doesn't exist (nil)
*	Array	*2\r\n$3\r\nfoo\r\n...	List of elements

How 'SET mykey myvalue' Is Encoded

```
*3\r\n        <-- array of 3 elements
$3\r\n        <-- first element is 3 bytes
SET\r\n       <-- "SET"
$5\r\n        <-- second element is 5 bytes
mykey\r\n     <-- "mykey"
$7\r\n        <-- third element is 7 bytes
myvalue\r\n   <-- "myvalue"
```

Parser Logic

```
std::vector<std::string> parseRespCommand(const std::string &input) {
    // If input starts with '*', parse as RESP:
    //   1. Read '*N' -> number of elements
    //   2. For each element:
    //      a. Read '$N' -> length of next string
    //      b. Read exactly N bytes -> that's the token
    //   3. Return vector of tokens
    // Otherwise: fall back to whitespace splitting
}
```

5. Concurrency & Thread Safety

Thread-per-Client Model

```
Main Thread
    |
    +-- accept() -> Client 1 Thread -> recv/process/send loop
    +-- accept() -> Client 2 Thread -> recv/process/send loop
    +-- accept() -> Client 3 Thread -> recv/process/send loop
    +-- ...
```

Thread Spawning Code

```
while (running) {
    int client_socket = accept(server_socket, nullptr, nullptr);
    clientThreads.emplace_back([client_socket, &cmdHandler]() {
        char buffer[1024] = {0};
        while (true) {
            memset(buffer, 0, sizeof(buffer));
            int bytes = recv(client_socket, buffer, sizeof(buffer)-1, 0);
            if (bytes <= 0) break;  // client disconnected
            std::string request(buffer, bytes);
            std::string response = cmdHandler.processCommand(request);
            send(client_socket, response.c_str(), response.size(), 0);
        }
        close(client_socket);
    });
}
```

Lambda Captures

client_socket: Captured by value -- each thread gets its own copy of the socket fd

&cmdHandler: Captured by reference -- all threads share the same handler (stateless, safe)

Mutex and Synchronization

Multiple threads accessing the same data simultaneously causes race conditions. Solution: std::mutex + std::lock_guard (RAII pattern).

```
bool RedisDatabase::set(const std::string& key, const std::string& value) {
    std::lock_guard<std::mutex> lock(db_mutex);  // LOCK acquired
    kv_store[key] = value;
    return true;
}   // lock_guard destructor -> UNLOCK (even if exception thrown)
```

Every single database method starts with std::lock_guard<std::mutex> lock(db_mutex). This means: only one thread modifies the DB at a time, reads are also serialized, and no data corruption is possible.

Thread Safety Matrix

	Client Thread 1	Client Thread 2	Persistence Thread
Reads kv_store	Must lock	Must lock	Must lock
Writes kv_store	Must lock	Must lock	Never writes

Reads running flag	Via atomic	Via atomic	N/A

6. Design Patterns

Singleton Pattern (RedisDatabase)

Ensures exactly ONE database instance shared across all threads and all client connections.

```
class RedisDatabase {
public:
    static RedisDatabase& getInstance();  // only way to get instance
private:
    RedisDatabase() = default;            // private constructor
    ~RedisDatabase() = default;
    RedisDatabase(const RedisDatabase&) = delete;            // no copy
    RedisDatabase& operator=(const RedisDatabase&) = delete; // no assign
};

RedisDatabase& RedisDatabase::getInstance() {
    static RedisDatabase instance;  // created once, thread-safe in C++11+
    return instance;
}
```

static local: In C++11+, function-local static initialization is guaranteed thread-safe. First thread creates, others wait.

delete copy: Prevents accidental second copy at compile time.

Signal Handling (Graceful Shutdown)

```
static RedisServer* globalServer = nullptr;

void signalHandler(int signum) {
    if (globalServer) {
        globalServer->shutdown();  // sets running=false, closes socket
    }
    exit(signum);
}

// In constructor:
signal(SIGINT, signalHandler);   // Ctrl+C
signal(SIGTERM, signalHandler);  // kill command
```

Shutdown Flow

```
1. OS delivers SIGINT (Ctrl+C)
2. signalHandler() called
3. shutdown() sets running=false, closes server_socket
4. accept() returns -1 -> loop exits
5. Server joins all client threads
6. Server dumps database to disk
7. Process exits
```

Global pointer: Signal handlers are C-style function pointers -- can't be member functions or lambdas. Global pointer is the bridge.

std::atomic<bool>: The running flag is read by main thread and written by signal handler. atomic prevents compiler optimizations that could cache stale values.

7. Data Structures

The Three Stores

```
// KEY-VALUE: string -> string
std::unordered_map<std::string, std::string> kv_store;
// {"session:123" -> "user_data", "config:timeout" -> "30"}

// LISTS: key -> ordered sequence of strings
std::unordered_map<std::string, std::vector<std::string>> list_store;
// {"job_queue" -> ["task1", "task2", "task3"]}

// HASHES: key -> field/value map (nested hash map)
std::unordered_map<std::string, std::unordered_map<std::string, std::string>> hash_store;
// {"user:1000" -> {"name" -> "Alice", "age" -> "30"}}

// EXPIRY: key -> deadline
std::unordered_map<std::string, std::chrono::steady_clock::time_point> expire_map;
```

Why unordered_map?

Operation	unordered_map	std::map
Lookup	O(1) average	O(log n)
Insert	O(1) average	O(log n)
Delete	O(1) average	O(log n)

For a cache, O(1) lookup is critical. We don't need sorted keys, so unordered_map (hash table) is the right choice.

Why vector for Lists? (Known Limitation)

std::vector gives O(1) access by index and O(1) push_back. However, LPUSH (insert at front) is O(n) because it shifts all elements. A std::deque would be O(1) for both ends.

Expiry Mechanism (Passive)

```
// Setting expiry:
expire_map[key] = steady_clock::now() + chrono::seconds(seconds);

// Checking (on GET):
auto exp_it = expire_map.find(key);
if (exp_it != expire_map.end()) {
    if (steady_clock::now() > exp_it->second) {
        kv_store.erase(key);
        list_store.erase(key);
        hash_store.erase(key);
        expire_map.erase(exp_it);
        return false;  // key expired
    }
}
```

steady_clock: Monotonic clock -- never jumps backwards even if system time changes. Essential for reliable timeouts.

8. Class-by-Class Breakdown

RedisServer (19 + 119 lines)

```
class RedisServer {
public:
    RedisServer(int port);
    void run();       // main server loop
    void shutdown();  // graceful shutdown
private:
    int port;
    int server_socket;
    std::atomic<bool> running;
    void setupSignalHandlers();
};
```

RedisDatabase: 26 Methods (80 + 454 lines)

Common (1)

Method	Return	Description
flushAll()	bool	Clears ALL stores and expire_map

Key-Value (7)

Method	Return	Description
set(key, value)	bool	Store string value
get(key, &value)	bool	Retrieve value (checks expiry first)
keys()	vector	All keys from all three stores
type(key)	string	"string", "list", "hash", or "none"
del(key)	bool	Remove from all stores
expire(key, secs)	bool	Set TTL
rename(old, new)	bool	Rename key in whichever store it lives

List (9)

Method	Return	Description
llen(key)	int	List length, -1 if not found
lpush(key, values)	int	Push to front, returns new length
rpush(key, values)	int	Push to back, returns new length
lpop(key, &value)	bool	Remove and return front element
rpop(key, &value)	bool	Remove and return back element
lrem(key, cnt, val)	int	Remove matching elements
lrange(key,s,e,&r)	bool	Get range (supports negative indices)
lindex(key, idx, &v)	bool	Get element by index
lset(key, idx, val)	bool	Set element at index

Hash (8)

Method	Return	Description
hset(key, fld, val)	int	Set field (1=new, 0=updated, -1=wrong type)
hget(key, fld, &v)	bool	Get field value
hexists(key, fld)	bool	Check if field exists
hdel(key, fld)	bool	Delete field
hgetall(key)	vec<pair>	All field-value pairs
hkeys(key)	vector	All field names
hvals(key)	vector	All field values
hlen(key)	int	Number of fields

Persistence (2)

Method	Return	Description
dump(filename)	bool	Serialize all data to file
load(filename)	bool	Deserialize from file, replacing all data

Type Safety Across Stores

```
int RedisDatabase::lpush(const std::string& key, ...) {
    std::lock_guard<std::mutex> lock(db_mutex);
    if (kv_store.count(key) || hash_store.count(key))
        return -1;  // WRONGTYPE error
    // ...
}
```

Prevents SET x "hello" then LPUSH x "oops" -- returns a type error, just like real Redis.

RedisCommandHandler (12 + 510 lines)

```
std::string processCommand(const std::string& commandLine) {
    auto tokens = parseRespCommand(commandLine);
    std::string cmd = tokens[0];
    std::transform(cmd.begin(), cmd.end(), cmd.begin(), ::toupper);

    RedisDatabase& db = RedisDatabase::getInstance();

    if      (cmd == "PING")  return handlePing(tokens, db);
    else if (cmd == "SET")   return handleSet(tokens, db);
    else if (cmd == "GET")   return handleGet(tokens, db);
    // ... 24 more commands ...
    else return "-Error: Unsupported command\r\n";
}
```

9. Complete Command Reference

Common Commands

Command	Example	Response	Purpose
PING	PING	PONG	Test connectivity
ECHO	ECHO "hello"	"hello"	Echo back message
FLUSHALL	FLUSHALL	OK	Delete ALL data

Key/Value Commands

Command	Example	Response	Notes
SET	SET k v	OK	Store string value
GET	GET k	"v" or (nil)	Checks expiry first
KEYS	KEYS *	1) "k"	Pattern arg accepted, not filtered
TYPE	TYPE k	string/list/hash/none	Returns data type
DEL/UNLINK	DEL k	(integer) 1 or 0	Remove from any store
EXPIRE	EXPIRE k 10	(integer) 1	Set TTL in seconds
RENAME	RENAME a b	OK	Rename without data loss

List Commands

Command	Example	Response	Notes
RPUSH	RPUSH L a b c	(integer) 3	Append to end
LPUSH	LPUSH L x	(integer) 4	Prepend to front
LGET	LGET L	1) x 2) a ...	All elements (custom cmd)
LLEN	LLEN L	(integer) 4	List length
LPOP	LPOP L	"x"	Remove+return first
RPOP	RPOP L	"c"	Remove+return last
LINDEX	LINDEX L 0	"a"	Supports negative indices
LSET	LSET L 1 new	OK	Replace at index
LREM	LREM L 2 x	(integer) 2	Remove N occurrences
LRANGE	LRANGE L 0 -1	1) a 2) b	Range with neg indices

Hash Commands

Command	Example	Response	Notes
HSET	HSET u name A	(integer) 1	1=new, 0=updated
HGET	HGET u name	"A"	Single field value
HEXISTS	HEXISTS u name	(integer) 1	Check field exists
HDEL	HDEL u age	(integer) 1	Remove field
HGETALL	HGETALL u	name A email ..	Flat array output
HKEYS	HKEYS u	1) name 2) email	All field names
HVALS	HVALS u	1) A 2) a@x.com	All field values
HLEN	HLEN u	(integer) 2	Field count
HMSET	HMSET u k1 v1 k2 v2	OK	Multi-set (returns OK)

10. Persistence Format

Dump File Format (dump.my_rdb)

```
K session:123           <-- K = key-value type, then key name
user_data               <-- value on next line

L job_queue 3           <-- L = list type, key name, element count
task1                   <-- 3 list items follow
task2
task3

H user:1000 2           <-- H = hash type, key name, field count
name Alice              <-- 2 field-value pairs follow (space-separated)
email a@x.com
```

Dump Algorithm

- Lock the mutex (no writes during dump)
- Iterate kv_store -> write 'K <key>\n<value>\n'
- Iterate list_store -> write 'L <key> <count>\n' then each item
- Iterate hash_store -> write 'H <key> <count>\n' then each field-value pair

Load Algorithm

- Lock the mutex
- Clear all stores
- Read line by line
- First character determines type (K/L/H)
- Parse key, count, and elements accordingly

When Persistence Happens

Trigger	Mechanism
Background thread	Dumps every 300 seconds (5 minutes)
On shutdown	Dumps before process exits (via signal handler)
On startup	Loads from file if it exists

11. Complete Data Flow

End-to-End: SET mykey myvalue

```
redis-cli                    Network              Server
----------                   -------              ------
1. User types:
   SET mykey myvalue

2. redis-cli encodes         ---------->  3. recv() reads bytes
   to RESP:                                  into buffer[1024]
   *3\r\n$3\r\nSET\r\n
   $5\r\nmykey\r\n                       4. processCommand()
   $7\r\nmyvalue\r\n                        |
                                            +- parseRespCommand()
                                            |  -> ["SET","mykey","myvalue"]
                                            +- cmd = "SET" (uppercased)
                                            +- handleSet(tokens, db)
                                            |  +- db.set("mykey","myvalue")
                                            |  |  +- lock_guard locks
                                            |  |  +- kv_store["mykey"]="myvalue"
                                            |  |  +- lock_guard unlocks
                                            |  +- return "+OK\r\n"
                                         5. send() sends "+OK\r\n"
6. redis-cli decodes         <----------
   "+OK\r\n" -> prints OK
```

Build System (Makefile)

Flag	Purpose
-std=c++17	C++17 standard (structured bindings, if constexpr)
-Wall	Enable all compiler warnings
-pthread	Link POSIX threads library
-MMD -MP	Auto-generate dependency files for header changes
-O2	Optimization level 2 (faster executable)

```
make          # compile and link
make clean    # remove build/ and executable
make rebuild  # clean + build
make run      # build then run
```

12. Testing

Automated Test Suite

```
./my_redis_server 6399 &
python3 test.py
# Runs 38 tests covering all commands via raw TCP + RESP
```

Manual Testing

```
make run                    # Terminal 1
./redis-cli -p 6379        # Terminal 2
```

Edge Cases to Verify

Test	Command	Expected
GET nonexistent	GET ghost	(nil)
DEL nonexistent	DEL ghost	(integer) 0
Wrong type	SET x str; LPUSH x val	Error
Out of range	LINDEX L 9999	(nil)
Expire then GET	SET k v; EXPIRE k 1; wait; GET k	(nil)
Persistence	SET data; restart; GET data	Original value

13. Limitations & Future Improvements

Issue	Impact	Fix
Thread-per-client	Dies at ~10K connections	Use epoll event loop
Single global mutex	All operations serialized	Sharded/per-key locks or shared_mutex
LPUSH is O(n)	Slow for large lists	Replace vector with deque
1024-byte buffer	Large commands truncated	Dynamic buffer allocation
Passive expiry only	Expired keys waste memory	Background expiry sweep
No memory limit	Grows until OOM	maxmemory + LRU eviction
KEYS pattern ignored	Always returns all keys	Implement glob pattern matching
No SET EX support	Can't set value + TTL atomically	Parse EX/PX options in SET

Recommended Next Steps

- 1. Replace std::vector with std::deque for lists
- 2. Fix DEL/EXPIRE/RENAME to return integer replies (: not +)
- 3. Add SET key value EX seconds support
- 4. Implement glob pattern matching for KEYS
- 5. Add background expiry thread
- 6. Replace thread-per-client with epoll
- 7. Add LRU eviction with memory cap
- 8. Add PUB/SUB support

14. Glossary

TCP:	Transmission Control Protocol -- reliable, ordered byte stream over a network
Socket:	OS-level endpoint for network communication
RESP:	REdis Serialization Protocol -- the wire format Redis uses
Mutex:	Mutual Exclusion -- a lock ensuring one thread accesses a resource at a time
RAII:	Resource Acquisition Is Initialization -- cleanup tied to object lifetime
Singleton:	Design pattern ensuring exactly one instance of a class exists
Thread safety:	Code works correctly when accessed by multiple threads simultaneously
TTL:	Time To Live -- seconds until a key expires
Atomic:	Operation that completes entirely or not at all, no intermediate state
epoll:	Linux kernel mechanism for efficiently monitoring many sockets
LRU:	Least Recently Used -- eviction policy removing oldest-accessed items
O(1):	Constant time -- operation speed doesn't depend on data size
Race condition:	Bug where behavior depends on unpredictable thread timing
Serialization:	Converting in-memory data to a byte stream for storage/transfer
Backlog:	Pending connections the OS queues before refusing new ones
SIGINT:	Signal sent by OS when user presses Ctrl+C
Detached thread:	Thread that runs independently -- parent doesn't wait for it
Wire protocol:	Exact format of bytes exchanged between client and server